# Full Stack Engineer Hiring Challenge

## Overview

In this challenge, you are asked to create an MVP Course Learning Platform, similar to a typical online learning platform.

It does not need to be perfect; what is more important is a neat code structure, core features that work well, and how you solve problems.

Deadline: 7 days from the date the challenge is given.

## Tech Stack

Frontend

- Next.js 15 (App Router)
- React 19
- TypeScript (strict mode)
- Tailwind CSS

Backend

- PostgreSQL
- Drizzle ORM
- Next.js Server Actions (more preferred than route handlers)

Authentication

- NextAuth.js v4

Library (optional)

- react-hot-toast
- date-fns
- shadcn/ui / Radix UI / custom components
- Video.js

Package Manager

- pnpm (recommended), npm, atau yarn

## Fitur yang Perlu Dibuat

1. Authentication

- User register (email & password)

- Login
- Logout
- Protected routes
- User profile page (view & edit)

Role user:

- student
- admin

2. Course Management

Struktur course:

Course

└ Chapter

  └ Lesson

Course field minimum:

- title
- description
- thumbnail
- slug
- status (draft / published)

Chapter:

- title
- description
- course_id
- sort order

Lesson:

- title
- description
- lesson type (video / text / quiz)
- video_url atau content
- sort order

minimum page:

- Course listing

- Course detail
- Lesson page
- Admin page untuk create/edit/delete course

3. User Progress Tracking

- Marking lessons as completed
- Viewing course progress
- Navigating to the next lesson

4. Quiz System

- The administrator can create quiz questions
- Users answer the quiz
- The system calculates the score
- Passing score: 70%
- Quiz result are saved

5. Subscription / Payment

- Subscription plan (Monthly / Yearly)
- Premium courses can only be accessed by users with an active subscription
- Payment should be integrated using stripe (sandbox) or Midtrans (sandbox)

6. Access Control

- Free courses can be accessed by all users.
- Premium courses are only accessible to users with an active subscription.
- If a user has not subscribed, display a prompt encouraging them to subscribe.

7. UI / UX

- Clean and easy-to-use interface.
- Fully responsive (mobile-friendly).
- Proper loading states.
- Clear and user-friendly error handling.

## Database

Use **Drizzle ORM** to define schema and manage migrations (recommended).

Expected Main Entities:

- Users
- Courses
- Chapters

- Lessons
- UserProgress
- Quizzes
- QuizAttempts
- Plans
- Subscriptions

Ensure that database relationships are clearly defined and use proper foreign keys.

## Submission

Please submit the challenge as a **GitHub repository**.

The repository must include:

README.md

- Brief project description
- Instructions to run the project locally
- Required environment variables
- Instructions to run migrations
- Instructions to seed data (admin user and sample course)

File tambahan

- .env.example
- Drizzle migration files

Make sure the project can be run locally without errors.

## Kriteria Penilaian

Code Quality

- Clean code structure
- Consistent coding style
- Proper use of TypeScript

Functionality

- Core features work properly
- Proper error handling
- Handles edge cases

Technical Implementation

- Well-designed database schema
- Proper use of Next.js features
- Authentication implementation

UI / UX

- Clean interface
- Responsive design
- Good user experience

Documentation

- README jelas
- Easy to follow setup instructions

## Bonus (Optional)

If time permits, you may add additional features such as:

- Course search / filtering
- Course rating or reviews
- Email notifications
- Admin analytics dashboard
- Video progress tracking
- Dark mode
- Unit tests or E2E tests
- Docker setup

These features are not required but will be considered a plus.